Prompt Engineering for Document Question Answering

Structuring instructions and evidence for reliable responses

Purpose

This short reference note is written for learning, semantic retrieval, and grounded question-answering experiments. It uses original explanatory content and is organized into coherent sections rather than unrelated filler.

1. What a Prompt Controls

A prompt communicates the task, constraints, available evidence, and desired output. It can clarify that retrieved text is evidence rather than an instruction, specify how to handle missing information, and request source references. Prompting cannot create facts absent from the context, and it does not guarantee that a model will follow every constraint.

2. Separate Instructions from Evidence

Use clear sections for system-level rules, the user's question, and retrieved passages. Label passages with source identifiers. This structure reduces ambiguity about which text is a task instruction and which text is content to analyze. Treat document contents as untrusted data: a PDF may contain text that looks like an instruction, but the application should not let source text override the intended task rules.

3. A Grounded Answer Pattern

A useful pattern is: answer using the supplied context; cite the relevant source labels; if the context does not support an answer, say that the documents do not provide enough information; do not invent citations. For complex questions, ask the model to distinguish directly stated facts from reasonable inferences. The application should still verify that returned source labels exist and point to actual retrieved chunks.

4. Output Formats

A consistent output format can make downstream display and testing easier. For example, request an answer field and a list of sources, or require short paragraphs followed by source labels. Structured output can reduce parsing errors but should be validated. If a model returns malformed JSON or references an unknown chunk ID, handle that case gracefully rather than trusting the output blindly.

5. Prompt Injection in Documents

A document may contain malicious or irrelevant instructions such as requests to reveal secrets or ignore prior rules. In document QA, retrieved passages should be treated as data to summarize or answer from, not as privileged instructions. Avoid placing secrets in prompts, restrict tools and permissions, and test with documents containing instruction-like text. Prompt wording is only one layer of defense; system design matters too.

6. Iterative Prompt Testing

Change one important prompt element at a time and run the same evaluation questions. Check factual correctness, grounding, abstention, source validity, and response length. Include difficult cases, conflicting passages, and irrelevant retrievals. Avoid judging a prompt only by whether one answer sounds better; measure its effects across the evaluation set.

Key Takeaways

Effective document-QA prompts clearly separate instructions, questions, and evidence. They define how to cite sources and handle missing information, while application-level validation remains necessary.

End of document • Use source passages as evidence and verify claims during evaluation.